```
(*
                            CS 51 Lab 19
                Conway's Game of Life Cellular Automaton
 *)
(*
                                SOLUTION
 *)
module G = Graphics ;;

(* Automaton parameters *)
let cGRID_SIZE = 100 ;;      (* width and height of grid in cells *)
let cSPARSITY = 5 ;;         (* inverse of proportion of cells initially live *)
let cRANDOMNESS = 0.00001 ;; (* probability of randomly modifying a cell *)

(* Rendering parameters *)
let cCOLOR_LIVE =   G.rgb  93  46  70 ;;  (* color to depict live cells *)
let cCOLOR_DEAD =   G.rgb 242 227 211 ;;  (* background color *)
let cCOLOR_LEGEND = G.rgb 173 106 108 ;;  (* color for textual legend *)
let cSIDE = 8 ;;             (* width and height of cells (in pixels) *)
let cRENDER_FREQUENCY = 1    (* how frequently grid is rendered (in ticks) *) ;;

(* Font specification for rendering the legend. OCaml font handling is
   platform dependent, so we leave this as `None`, but we provide the
   functionality for use in the solution set. Feel free to play with
   this or leave it as is. *)
let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"


(*----------------------------------------------------------------------
  The Game of Life
 *)

(*......................................................................
  Game of Life states and updates
 *)

(* We take the opportunity to abstract the cell states as an
   enumerated type *)
type life_state =
  | Live
  | Dead ;;

(* flipped state -- Returns the opposite state to `state`. *)
let flipped (state : life_state) : life_state =
  match state with
  | Live -> Dead
  | Dead -> Live ;;

(* offset index off -- Returns the `index` offset by `off` allowing
   for wraparound. *)
let rec offset (index : int) (off : int) : int =
  if index + off < 0 then
```

```
    cGRID_SIZE - (offset ~-index ~-off)
  else
    (index + off) mod cGRID_SIZE ;;

(* life_update grid i j -- Returns the updated value for cell at `i,
   j` in the grid based on rules of Conway's Game of Life. In this
   implementation, after updating as per the standard GoL update rule,
   a cell's state is then flipped with probability given by
   cRANDOMNESS. *)
let life_update (grid : life_state array array) (i : int) (j : int)
              : life_state =

  (* We give a few variant update rules for the Game of Life. These
     lists are indexed by adjacency count to give the generated cell
     status.
                          0     1     2     3     4     5     6     7     8
                          |     |     |     |     |     |     |     |     |
                          v     v     v     v     v     v     v     v     v   *)
  (* B3/S23: Conway's original game of life *)
     let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Dead; Dead; Dead] in
     let live_update = [Dead; Dead; Live; Live; Dead; Dead; Dead; Dead; Dead] in

  (* B3/S12345: https://conwaylife.com/wiki/OCA:Maze
     let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Dead; Dead; Dead] in
     let live_update = [Dead; Live; Live; Live; Live; Live; Dead; Dead; Dead] in
   *)
  (* B36/S125: https://conwaylife.com/wiki/OCA:2×2
     let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Live; Dead; Dead] in
     let live_update = [Dead; Live; Live; Dead; Dead; Live; Dead; Dead; Dead] in
   *)
  (* B3678/34678: https://conwaylife.com/wiki/OCA:Day_%26_Night
     let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Live; Live; Live] in
     let live_update = [Dead; Dead; Dead; Live; Live; Dead; Live; Live; Live] in
   *)

  let neighbors = ref 0 in
  for i' = ~-1 to ~+1 do
    for j' = ~-1 to ~+1 do
      let i_ind = offset i i' in
      let j_ind = offset j j' in
      neighbors := !neighbors
                   + match grid.(i_ind).(j_ind) with
                     | Live -> 1
                     | Dead -> 0
    done
  done;
  (* determine new state based on neighbor count *)
  let new_state =
    match grid.(i).(j) with
    | Live -> List.nth live_update (!neighbors - 1)
    | Dead -> List.nth dead_update !neighbors in
  (* randomly flip an occasional cell state *)
  if Random.float 1.0 <= cRANDOMNESS then
    flipped new_state
```

```
  else
    new_state ;;

(* life_initialize grid -- Fills `grid` with a random mix of Live and
   Dead cells, with roughly 1-in-cSPARSITY cells initially live. *)
let life_initialize (grid : life_state array array) : unit =
  for i = 0 to cGRID_SIZE - 1 do
    for j = 0 to cGRID_SIZE - 1 do
      grid.(i).(j) <- if Random.int cSPARSITY = 0 then Live else Dead
    done
  done

(*.....................................................................
  Making the Game of Life
 *)

(* GoL automaton specification *)

module LifeSpec : (Cellular.AUT_SPEC
                   with type state = life_state) =
  struct
    type state = life_state
    let initial : state = Dead
    let grid_size : int = cGRID_SIZE
    let initialize = life_initialize
    let update = life_update
    let name : string = "Conway's Life"
    let cell_color (cell : state) : G.color =
      match cell with
      | Live -> cCOLOR_LIVE
      | Dead -> cCOLOR_DEAD
    let side_size : int = cSIDE
    let legend_color : G.color = cCOLOR_LEGEND
    let font : string option = cFONT
    let render_frequency : int = cRENDER_FREQUENCY
  end ;;

(* GoL automaton *)

module Aut = Cellular.Automaton (LifeSpec) ;;

(*.....................................................................
  Running the automaton and displaying the results
 *)

(* main seed -- Runs the automaton using the provided random `seed`
   (for replicability), displaying updates to the graphics window. *)
let main seed =

  Random.init seed;      (* initialize randomness *)
  Aut.graphics_init ();  (* ... and graphics window *)

  Aut.run_grid (Aut.initialized_grid ()) ;;

```

```
(* Run the automaton with user-supplied seed *)
let _ =
  if Array.length Sys.argv <= 1 then
    Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
                  Sys.argv.(0)
  else
    main (int_of_string Sys.argv.(1)) ;;

```